When it was introduced, a key feature of the `[ApiController]` attribute was the Problem Details support, but as I described in chapter 5, the same automatic conversion to Problem Details is now supported by the default `ExceptionHandlerMiddleware` and `StatusCodePagesMiddleware.` Nevertheless, the `[ApiController]` conventions can significantly reduce the amount of boilerplate code you have to write and ensure that validation failures are handled automatically, for example.

As is common in ASP.NET Core, you will be most productive if you follow the conventions rather than trying to fight them. However, if you don't like some of the conventions introduced by `[ApiController]`, or want to customize them, you can easily do so.

You can customize the web API controller conventions your application uses by calling `ConfigureApiBehaviorOptions()` on the `IMvcBuilder` object returned from the `AddControllers()` method in your Program.cs file. For example, you could disable the automatic `400` responses on validation failure, as shown in the following listing.

**Listing 20.12 Customizing `[ApiAttribute]` behaviors**

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
    .ConfigureApiBehaviorOptions(options =>                ❶
    {
        options.SuppressModelStateInvalidFilter = true;    ❷
    });

// ...
```

❶ Controls which conventions are applied by providing a configuration lambda

❷ This would disable the automatic 400 responses for invalid requests.